Abstract

This report is all about checking out how well different methods work for figuring out if Yelp reviews are positive or negative, and what people are talking about in those reviews. I wanted to see if newer, fancy computer models are better than the older, simpler ones at guessing the feelings in restaurant reviews. First, I cleaned up the review text by doing things like fixing typos, making everything lowercase, getting rid of unnecessary characters, and simplifying words. Then, I used the cleaned-up text with some basic computer models like Logistic Regression, Naive Bayes, XGBoost, Support Vector Machine (SVM), and Random Forest. I even tweaked a couple of them (SVM and Random Forest) to work their best.

At the same time, I tried out some of the more advanced models called BERT, DistilBERT, and RoBERTa. These models are like smarty-pants that have already learned a lot from other text, and I taught them how to be good at understanding Yelp reviews. Turns out, these newer models did a way better job than the older ones, especially BERT and RoBERTa which were the most accurate. I also tried to figure out the main topics in the reviews using methods called LDA, NMF, and LSA. I even made some graphs and charts to help see how the models worked and what topics they found.

Based on what I saw, cleaning the text well is super important, and those newer models are really good at understanding language. I wrap up by talking about what this all means, where I could have done better, and what I could try in the future, like using more data, different versions of the models, or cooler ways to find topics. Think of this as a straightforward how-to for using new-school computer methods to understand feelings and topics in real-world text.

Table of Contents

Abstract **3**

Introduction **6**

Task 1: Preprocess the Textual Data **7**

1.1 Dataset Overview *7*

I. **Dataset Shape and Structure:** 7
II. **Dataset Summary:** 8
III. **Data Integrity:** 8

1.2 Preprocessing Pipeline *9*

I. **Contraction Expansion** 9
II. **Lowercasing** 9
III. **Punctuation and Number Removal** 9
IV. **Tokenization** 10
V. **Stopword Removal** 10
VI. **Lemmatization** 10
VII. **Whitespace Normalization** 11

Task 2: Classification Using Bag-of-Words Representation **12**

2.1 Feature Extraction: Bag of Words *12*

2.2 Train-Test Split *12*

2.3 Classification Algorithms *12*

- **Logistic Regression:** 12
- **Naive Bayes:** 12
- **XGBoost:** 13
- **Support Vector Machine (SVM):** 13
- **Random Forest:** 13

2.4 Model Training and Hyperparameter Tuning *13*

2.5 Evaluation Metrics *14*

2.6 Results and Comparisons *14*

2.7 Classification reports and Confusion Matrices *15*

2.8 Comparison of Traditional Models *16*

2.9 Performance Visualization *17*

Task 3: Classification Using BERT-Based Models with Fine-Tuning **18**

- *Introduction to Transformer Models* *18*
- *Why Use Transformers?* *18*

3.1 BERT Fine-Tuning *18*

I. **Data Preparation and Train-Test Split** 18
II. **Tokenization** 18
III. **Dataset Construction** 19
IV. **Model Initialization and Fine-Tuning** 19
V. **Evaluation and Metrics** 19
VI. **Model Saving and Cleanup** 20

3.2 DistilBERT Fine-Tuning *21*
I. **Data Preparation and Train-Test Split** 21
II. **Tokenization** 21
III. **Dataset Construction** 21
IV. **Model Initialization and Fine-Tuning** 21
V. **Evaluation and Metrics** 22
VI. **Model Saving and Cleanup** 23

3.3 RoBERTa Fine-Tuning *23*
I. **Data Preparation and Train-Test Split** 23
II. **Tokenization** 24
III. **Dataset Construction** 24
IV. **Model Initialization and Fine-Tuning** 24
V. **Evaluation and Metrics** 25
VI. **Model Saving and Cleanup** 25

3.4 Model Comparison: Classical Machine Learning vs. Fine-Tuned Transformers *26*
I. **Performance Metrics Comparison** 26
II. **Accuracy vs. Run Time for Transformers** 27

Task 4: Topic Detection **28**

- **LDA - Latent Dirichlet Allocation** 28
- **NMF - Non-negative Matrix Factorization** 28
- **LSA - Latent Semantic Analysis** 28

4.1 Data Preparation for Topic Modeling *28*

4.2 Training the Topic Models *28*

4.3 Extracting and Interpreting Topics *29*

4.4 Visualizing Topics *30*

4.5 Document Clustering in Topic Space *31*

4.6 Differences Between Methods *31*

4.7 Findings *32*

References **33**

Introduction

These days, everyone's online, and sites like Yelp, TripAdvisor, and Google Reviews have changed how I pick what to buy and where to go. Restaurant reviews are super important because they really sway people's opinions. All this review text gives us tons of info about what customers think, but sorting through it can be tough.

Sentiment analysis, which is part of computer language stuff, figures out the mood of a text. It's a big deal for businesses that want to know if customers are happy, keep their image good, and stay ahead of the game. Also, finding the main subjects in the reviews helps us get why customers like or don't like things.

Old school machine learning like Logistic Regression, Naive Bayes, and Support Vector Machines have been used for a while to sort out texts. If you mix those with ways to pull out important features like Bag of Words or TF-IDF, you can do a decent job at sentiment analysis. But now, deep learning is changing things, especially with models like BERT, DistilBERT, and RoBERTa. These models are much better at understanding language and how words relate to each other.

This project compares the old machine learning methods with the new transformer models to see how well they do at figuring out the sentiment of Yelp restaurant reviews. First, I clean up the review text to make sure it's all the same format. Then, I train and test both types of models on the same info to see which does better. I also use topic modeling methods such as Latent Dirichlet Allocation (LDA), Non-negative Matrix Factorization (NMF), and Latent Semantic Analysis (LSA), to find the main subjects that people talk about in their reviews.

Task 1: Preprocess the Textual Data

Effective data preprocessing is fundamental to the success of any natural language processing (NLP) project. Raw textual data, especially user-generated content such as Yelp reviews, often contains inconsistencies, noise, and irrelevant information that can hinder the performance of machine learning models. Therefore, a systematic and thorough preprocessing pipeline was implemented to transform the raw reviews into a clean and structured format suitable for analysis.

1.1 Dataset Overview

The dataset used in this project consists of 1,000 restaurant reviews sourced from the publicly available "yelp_labelled.txt" file. Each review is accompanied by a binary sentiment label: 1 for positive sentiment and 0 for negative sentiment. The dataset is balanced, with an equal distribution of positive and negative reviews, which is advantageous for training and evaluating classification models.

I. Dataset Shape and Structure:

- Number of rows (reviews): 1,000
- Number of columns: 2

```
df.shape
(1000, 2)
```

- Column names:
  - text: The review text (string)
  - label: The sentiment label (integer; 1 = positive, 0 = negative)

```
df.columns
Index(['text', 'label'], dtype='object')
```

Sample Data:

```
df.head()
```

	text	label
0	Wow... Loved this place.	1
1	Crust is not good.	0
2	Not tasty and the texture was just nasty.	0
3	Stopped by during the late May bank holiday of...	1
4	The selection on the menu was great and so wer...	1

```
df.tail()
```

	text	label
995	I think food should have flavor and texture an...	0
996	Appetite instantly gone.	0
997	Overall I was not impressed and would not go b...	0
998	The whole experience was underwhelming, and I ...	0
999	Then, as if I hadn't wasted enough of my life ...	0

II. Dataset Summary:

- Total reviews: 1,000
- Unique reviews: 996

```
df.describe(include='object')
```

	text
count	1000
unique	996
top	I would not recommend this place.
freq	2

- Class distribution:
  - Positive (label = 1): 500
  - Negative (label = 0): 500

```
df.describe()
```

	label
count	1000.00000
mean	0.50000
std	0.50025
min	0.00000
25%	0.00000
50%	0.50000
75%	1.00000
max	1.00000

III. Data Integrity:

- No missing values in either column.
- Balanced classes ensure fair evaluation of classification models.

```
df.isnull().sum()

text     0
label    0
dtype: int64
```

1.2 Preprocessing Pipeline

I created a system to deal with some typical problems with text data. These problems include things like contractions, different capitalization styles, punctuation, numbers, common words that don't add much, and words that are slightly different but have the same basic meaning. Each review was put through these steps in order:

I. Contraction Expansion

In casual writing, people often use contractions like can't or won't, but they can make things a bit unclear. To fix this, I used a contractions tool to spell out all the contractions in the reviews. For example, can't became cannot. Doing this makes everything consistent and helps the next steps, like breaking down words and finding their root forms, work better.

```
--- After Expanding Contractions ---
Review 1: Wow... Loved this place.
Review 2: Crust is not good.
Review 3: Not tasty and the texture was just nasty.
Review 4: Stopped by during the late May bank holiday off Rick Steve recommendation and loved it.
Review 5: The selection on the menu was great and so were the prices.
```

II. Lowercasing

To keep things simple, all words were made lowercase. This way, the system doesn't see Good and good as things, shrinking the word list and making it easier to understand stuff in general.

```
--- After Converting to Lowercase ---
Review 1: wow... loved this place.
Review 2: crust is not good.
Review 3: not tasty and the texture was just nasty.
Review 4: stopped by during the late may bank holiday off rick steve recommendation and loved it.
Review 5: the selection on the menu was great and so were the prices.
```

III. Punctuation and Number Removal

Punctuation marks and numerical digits were removed from the text using regular expressions. Punctuation often does not contribute to the semantic meaning in sentiment analysis tasks, and numbers are typically irrelevant unless they carry specific contextual information, which is rare in restaurant reviews.

```
--- After Removing Numbers ---
Review 1: wow loved this place
Review 2: crust is not good
Review 3: not tasty and the texture was just nasty
Review 4: stopped by during the late may bank holiday off rick steve recommendation and loved it
Review 5: the selection on the menu was great and so were the prices
```

IV. Tokenization

The cleaned text was tokenized using the “RegexpTokenizer” from the NLTK library. Tokenization involves splitting the text into individual words or tokens, which serve as the basic units for further processing and feature extraction.

```
--- After Tokenization ---
Review 1: wow loved this place
Review 2: crust is not good
Review 3: not tasty and the texture was just nasty
Review 4: stopped by during the late may bank holiday off rick steve recommendation and loved it
Review 5: the selection on the menu was great and so were the prices
```

V. Stopword Removal

Stopwords are common words (such as "the", "is", "and") that gesnerally do not carry significant meaning and can introduce noise into the analysis. The NLTK stopword list for English was used to filter out these words from the tokenized reviews.

```
--- After Removing Stop Words ---
Review 1: wow loved place
Review 2: crust good
Review 3: tasty texture nasty
Review 4: stopped late may bank holiday rick steve recommendation loved
Review 5: selection menu great prices
```

VI. Lemmatization

Lemmatization reduces words to their base or dictionary form (lemma). For example, "running", "ran", and "runs" are all reduced to "run". This process, performed using the WordNetLemmatizer, helps in consolidating different forms of a word, thereby reducing dimensionality and improving model performance.

```
--- After Lemmatization ---
Review 1: wow loved place
Review 2: crust good
Review 3: tasty texture nasty
Review 4: stopped late may bank holiday rick steve recommendation loved
Review 5: selection menu great price
```

VII. Whitespace Normalization

After the above transformations, extra whitespace that may have resulted from the removal of punctuation or stopwords was eliminated. This step ensures that the final cleaned text is well-formatted and free of unnecessary spaces.

```
--- After Removing Extra Whitespace ---
Review 1: wow loved place
Review 2: crust good
Review 3: tasty texture nasty
Review 4: stopped late may bank holiday rick steve recommendation loved
Review 5: selection menu great price
```

Preprocessing Results

The effectiveness of the preprocessing pipeline can be illustrated by examining sample reviews before and after cleaning:

Original Review	After Preprocessing
Wow... Loved this place.	wow loved place
Crust is not good.	crust good
Not tasty and the texture was just nasty.	tasty texture nasty
The selection on the menu was great and so were the prices.	selection menu great price

I took the raw Yelp reviews and cleaned them up, putting them into a format that's easy to work with. Getting the data ready like this is super important if I want our machine learning models to do their best work later, whether those models are traditional or the transformer type.

Task 2: Classification Using Bag-of-Words Representation

2.1 Feature Extraction: Bag of Words

After preprocessing, the cleaned text data was transformed into a numerical format suitable for machine learning algorithms. The Bag of Words (BoW) model was employed for this purpose. BoW represents each document as a vector of word counts, disregarding grammar and word order but preserving multiplicity. This approach is simple yet effective for many text classification tasks.

The CountVectorizer from scikit-learn was used to convert the cleaned reviews into a document-term matrix. Each row in this matrix corresponds to a review, and each column represents a unique word from the entire corpus. The value in each cell indicates the frequency of the corresponding word in the review.

Vectorize Final Cleaned Text

```
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(final_cleaned)
y = df['label']
```

2.2 Train-Test Split

To see how well the model would do, the data was split into two groups for training and testing. Eighty percent of the data (800 reviews) was for training, and 20% (200 reviews) was for the test. The split was done to keep the number of good and bad reviews even in both groups.

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

2.3 Classification Algorithms

- **Logistic Regression:** A linear model commonly used for binary classification tasks. It estimates the probability that a given input belongs to a particular class.
- **Naive Bayes:** A probabilistic classifier based on Bayes' theorem, assuming independence between features. The Multinomial Naive Bayes variant is particularly suited for text data.

- **XGBoost:** An efficient and scalable implementation of gradient boosting, which builds an ensemble of weak learners (decision trees) to improve predictive accuracy.
- **Support Vector Machine (SVM):** A powerful classifier that finds the optimal hyperplane to separate classes. The LinearSVC implementation was used, with hyperparameter tuning via GridSearchCV.
- **Random Forest:** An ensemble method that constructs multiple decision trees and aggregates their predictions. Hyperparameters were optimized using GridSearchCV.

Define Default Models

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Naive Bayes": MultinomialNB(),
    "XGBoost": XGBClassifier(eval_metric='logloss')
}

results = {}
```

```
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
```

2.4 Model Training and Hyperparameter Tuning

Each model was trained on the training set and evaluated on the test set. For SVM and Random Forest, grid search was used to identify the best hyperparameters:

- **SVM:** Tuned parameters included the regularization parameter C and the maximum number of iterations.

GridSearch for SVM

```
print("\n Tuning Support Vector Machine (SVM)...")
svm_param_grid = {'C': [0.1, 1, 10], 'max_iter': [1000, 2000]}
svm_grid = GridSearchCV(LinearSVC(), svm_param_grid, cv=3, n_jobs=-1)
svm_grid.fit(X_train, y_train)
best_svm = svm_grid.best_estimator_
y_pred_svm = best_svm.predict(X_test)
```

- **Random Forest:** Tuned parameters included the number of estimators, maximum tree depth, and minimum samples required to split a node.

GridSearch for Random Forest

```
print("\n Tuning Random Forest...")
rf_param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}
rf_grid = GridSearchCV(RandomForestClassifier(random_state=42), rf_param_grid, cv=3, n_jobs=-1)
rf_grid.fit(X_train, y_train)
best_rf = rf_grid.best_estimator_
y_pred_rf = best_rf.predict(X_test)
```

2.5 Evaluation Metrics

Model performance was assessed using several standard metrics:

- **Accuracy:** The proportion of correctly classified reviews.
- **Precision:** The proportion of positive predictions that are actually positive.
- **Recall:** The proportion of actual positives that are correctly identified.
- **F1 Score:** The harmonic mean of precision and recall, providing a balanced measure.
- **Confusion Matrix:** A table showing the counts of true positives, true negatives, false positives, and false negatives.

```
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
results[name] = {"Accuracy": acc}
print(f" Accuracy: {acc:.4f}")
```

```
cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Negative", "Positive"])
disp.plot(cmap="Blues", values_format='d')
plt.title(f"Confusion Matrix – {name}")
plt.grid(False)
plt.show()
```

2.6 Results and Comparisons

Table summarizes the performance of each model on the test set:

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.77	0.83	0.70	0.76
Naive Bayes	0.75	0.76	0.76	0.76
XGBoost	0.69	0.81	0.54	0.65
SVM (Tuned)	0.76	0.80	0.72	0.76
Random Forest (Tuned)	0.69	0.81	0.54	0.65

“Logistic Regression” and “SVM” achieved the highest accuracy and F1 scores among the traditional models, indicating their effectiveness for this binary sentiment classification task. “Naive Bayes” performed slightly lower but remained competitive, reflecting its suitability for text data. “XGBoost” and “Random Forest” did not outperform the linear models, possibly due to the relatively small dataset size and the high dimensionality of the feature space.

2.7 Classification reports and Confusion Matrices

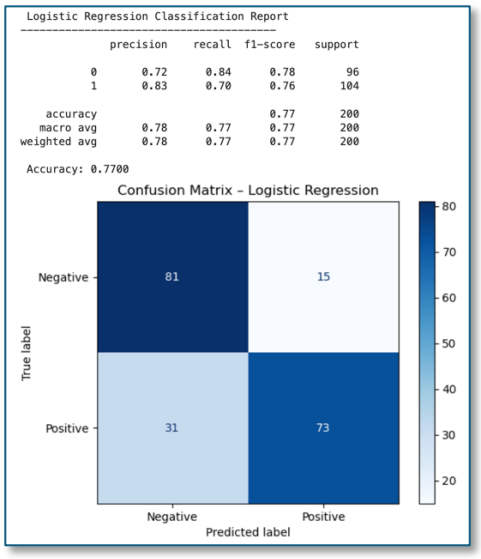
Classification reports and Confusion matrices were generated for each model to provide a detailed view of their classification performance.

<table>
<tr>
<td>

Logistic Regression Classification Report

	precision	recall	f1-score	support
0	0.72	0.84	0.78	96
1	0.83	0.70	0.76	104
accuracy			0.77	200
macro avg	0.78	0.77	0.77	200
weighted avg	0.78	0.77	0.77	200

Accuracy: 0.7700

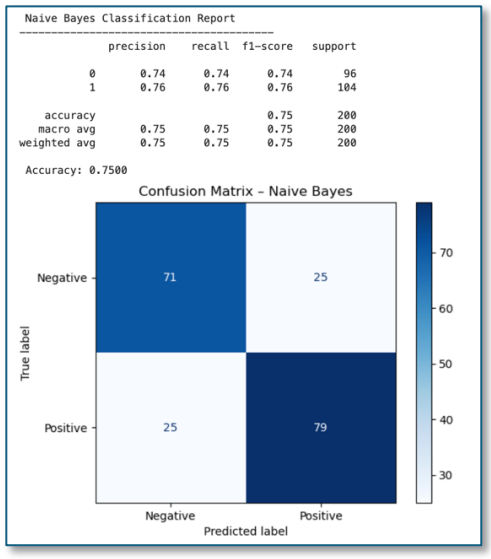


</td>
<td>

Naive Bayes Classification Report

	precision	recall	f1-score	support
0	0.74	0.74	0.74	96
1	0.76	0.76	0.76	104
accuracy			0.75	200
macro avg	0.75	0.75	0.75	200
weighted avg	0.75	0.75	0.75	200

Accuracy: 0.7500

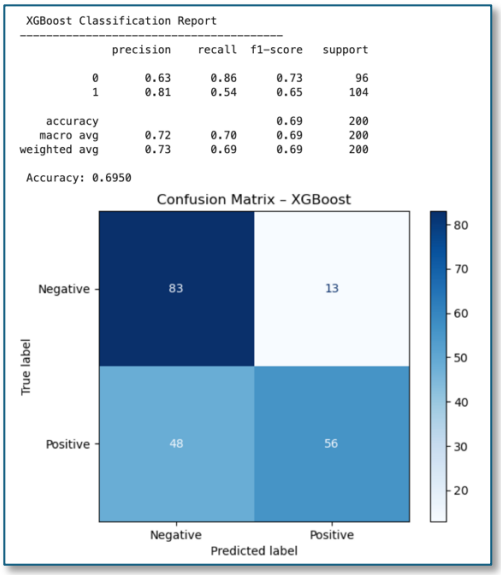


</td>
</tr>
<tr>
<td>This matrix shows that the model correctly identified 81 negative and 73 positive reviews, with some misclassifications.</td>
<td>This matrix shows that the model correctly identified 71 negative and 79 positive reviews, with some misclassifications.</td>
</tr>
<tr>
<td>

XGBoost Classification Report

	precision	recall	f1-score	support
0	0.63	0.86	0.73	96
1	0.81	0.54	0.65	104
accuracy			0.69	200
macro avg	0.72	0.70	0.69	200
weighted avg	0.73	0.69	0.69	200

Accuracy: 0.6950



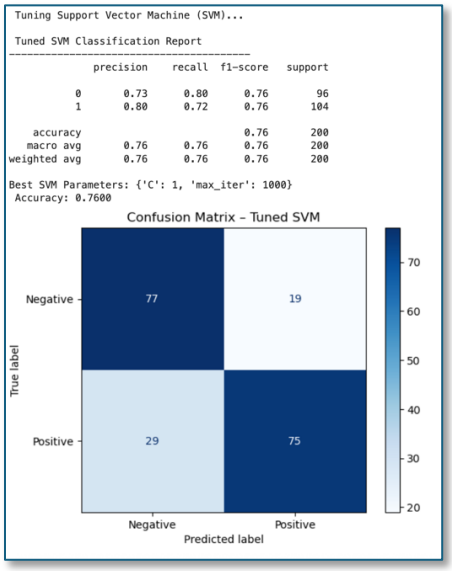
</td>
<td>

Tuning Support Vector Machine (SVM)...

Tuned SVM Classification Report

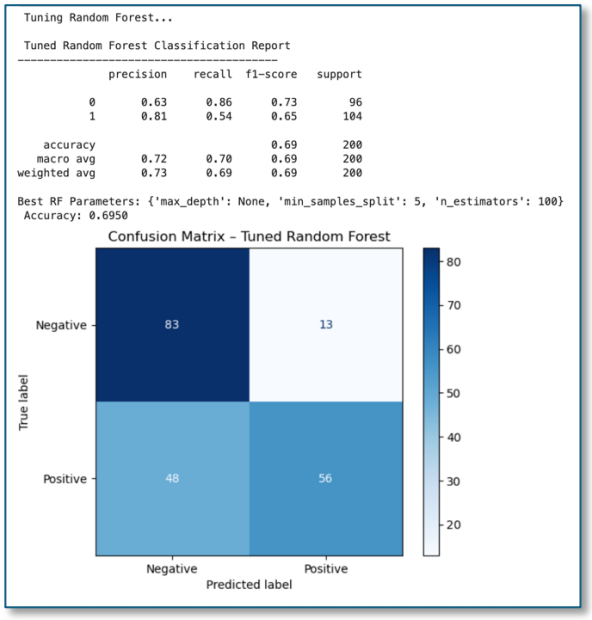
	precision	recall	f1-score	support
0	0.73	0.80	0.76	96
1	0.80	0.72	0.76	104
accuracy			0.76	200
macro avg	0.76	0.76	0.76	200
weighted avg	0.76	0.76	0.76	200

Best SVM Parameters: {'C': 1, 'max_iter': 1000}
Accuracy: 0.7600



</td>
</tr>
</table>

This matrix shows that the model correctly identified 83 negative and 56 positive reviews, with some misclassifications.	This matrix shows that the model correctly identified 77 negative and 75 positive reviews, with some misclassifications.



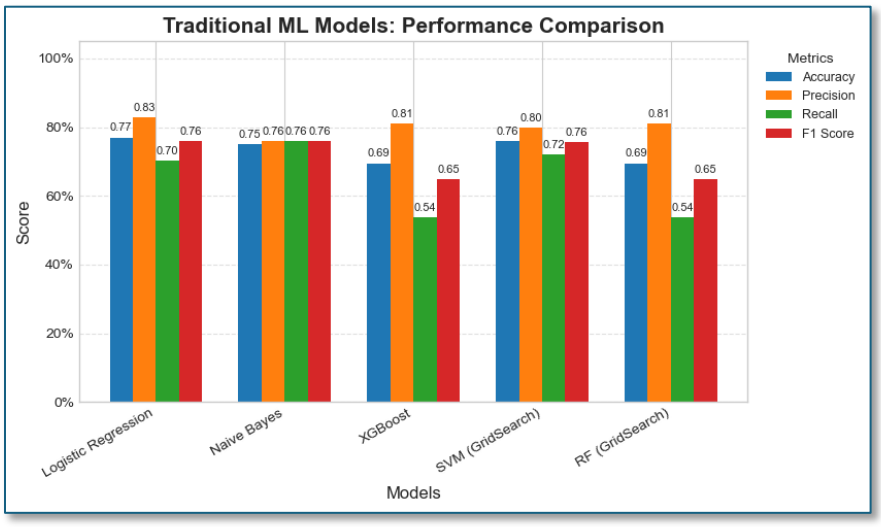
This matrix shows that the model correctly identified 83 negative and 56 positive reviews, with some misclassifications.

2.8 Comparison of Traditional Models

Overall, the simpler models like Logistic Regression and SVM turned out to be the most effective for this project, giving better accuracy and F1 scores than the more complex methods like Random Forest and XGBoost. This shows that when you have a smaller, well-prepared dataset, straightforward linear models can actually do a great job and are often easier to use. Naive Bayes also did reasonably well, proving itself as a reliable starting point for text classification. In this case, the more advanced ensemble models did not really offer any extra benefit, probably because the dataset was not large or complex enough for them to shine. These results suggest that sometimes, sticking with the basics is the best approach. With that in mind, it's interesting to see if the newer transformer-based models, which are designed to understand language in a much deeper way, can take things to the next level. That's what the next section will explore.

2.9 Performance Visualization

A grouped bar chart was created to compare the accuracy, precision, recall, and F1 score of all models. This visualization highlights the relative strengths and weaknesses of each approach and provides a clear overview of model performance.



Task 3: Classification Using BERT-Based Models with Fine-Tuning

▪ Introduction to Transformer Models

Recently, transformer models have really changed things in natural language processing. Unlike older machine learning methods that just counted words or used basic features, transformers try to get the meaning and how words relate to each other in a sentence. Models such as **BERT, DistilBERT,** and **RoBERTa** have pushed the limits for many NLP jobs, such as sentiment analysis, by using deep learning and attention tricks to understand the details of human language.

▪ Why Use Transformers?

Transformer models are great because they understand words by looking at how they're used, not just one by one. This really matters for things like figuring out if a review is positive or negative, where the feeling can come from small hints and groups of words. By training these models, which already have a good sense of language, on our set of Yelp reviews, we can do a lot even if we don't have tons of examples.

3.1 BERT Fine-Tuning

I. Data Preparation and Train-Test Split

The cleaned review texts and their sentiment labels were combined into a new DataFrame. The data was split into training and testing sets, with 80% used for training and 20% for testing, ensuring a balanced distribution of positive and negative reviews.

Combine final cleaned text with labels

```
df_bert = pd.DataFrame({'text': final_cleaned, 'label': df['label']})

train_texts, test_texts, train_labels, test_labels = train_test_split(
    df_bert['text'].tolist(), df_bert['label'].tolist(), test_size=0.2, random_state=42
)
```

II. Tokenization

The BERT tokenizer (bert-base-uncased) was used to tokenize the review texts. Each review was converted into a sequence of tokens, with padding and truncation applied to a maximum length of 128 tokens.

BERT Tokenizer

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

train_encodings = tokenizer(train_texts, truncation=True, padding=True, max_length=128)
test_encodings = tokenizer(test_texts, truncation=True, padding=True, max_length=128)
```

III. Dataset Construction

The tokenized data and labels were wrapped into a custom PyTorch Dataset class, enabling efficient data loading and batching during training and evaluation.

```
class YelpDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels
    def __getitem__(self, idx):
        return {key: torch.tensor(val[idx]) for key, val in self.encodings.items()} | {'labels': torch.tensor(self.labels[idx])}
    def __len__(self):
        return len(self.labels)

train_dataset = YelpDataset(train_encodings, train_labels)
test_dataset = YelpDataset(test_encodings, test_labels)
```

IV. Model Initialization and Fine-Tuning

The pre-trained BERT model for sequence classification was loaded and configured for binary sentiment analysis. Training arguments were set (3 epochs, batch size 16, evaluation per epoch), and the model was fine-tuned using the HuggingFace Trainer API.

BERT Fine-Tuning Model

```
bert_model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=64,
    eval_strategy="epoch",
    logging_dir="./logs",
    logging_steps=10
)
```

```
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    acc = accuracy_score(labels, preds)
    return {"accuracy": acc}
```

Create Trainer

```
trainer = Trainer(
    model=bert_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
```

BERT Fine-Tuning Training

```
trainer.train()
```

[150/150 01:03, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy
1	0.452000	0.453266	0.820000
2	0.234200	0.360145	0.850000
3	0.092300	0.405207	0.870000

```
TrainOutput(global_step=150, training_loss=0.34739349524180096, metrics={'train_runtime': 64.295, 'train_samples_per_second': 37.328, 'train_steps_per_second': 2.333, 'total_flos': 320666598720000.0, 'train_loss': 0.34739349524180096, 'epoch': 3.0})
```

V. Evaluation and Metrics

After training, the BERT model was evaluated on the test set. Metrics such as accuracy, precision, recall, and F1 score were calculated. A confusion matrix was generated to visualize the classification results.

- Accuracy: 87%
- Precision: 0.88
- Recall: 0.88

- F1 Score: 0.88

Evaluation

```
eval_result = trainer.evaluate()
run_time_bert = eval_result['eval_runtime']
accuracy_bert = eval_result['eval_accuracy']
results["BERT Fine-Tuned"] = {"Accuracy": accuracy_bert}
print(f"\n BERT Accuracy: {eval_result['eval_accuracy']:.4f}")
```

```
BERT Accuracy: 0.8700
```

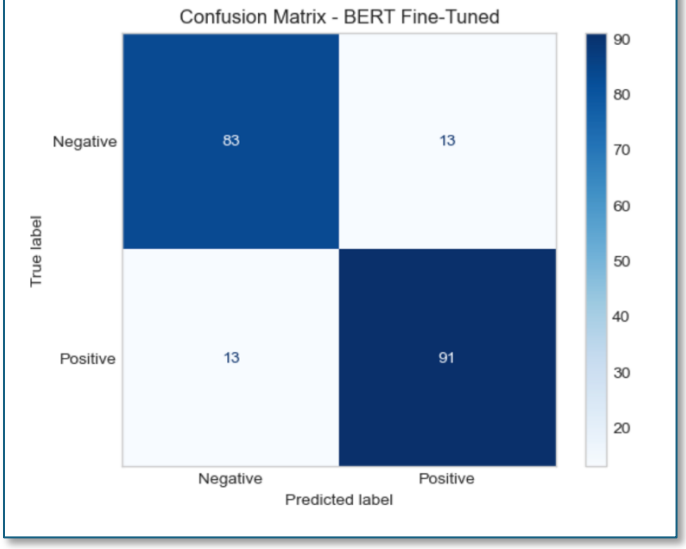
```
Classification Report for BERT Fine-Tuning:

              precision    recall  f1-score   support

    Negative       0.86      0.86      0.86        96
    Positive       0.88      0.88      0.88       104

    accuracy                           0.87       200
   macro avg       0.87      0.87      0.87       200
weighted avg       0.87      0.87      0.87       200

<Figure size 600x400 with 0 Axes>
```



VI. Model Saving and Cleanup

The fine-tuned BERT model and tokenizer were saved for future use. Memory was cleared to free up resources.

Save BERT Fine-Tuned model and tokenizer for future use

```
# Save model and tokenizer
trainer.save_model("./bert_saved_model")
tokenizer.save_pretrained("./bert_saved_model")

# Cleanup (free memory)
del trainer
del bert_model
del train_dataset, test_dataset
del train_encodings, test_encodings

# Clear CUDA cache (if using GPU)
if torch.cuda.is_available():
    torch.cuda.empty_cache()

# Force garbage collection
gc.collect()
```

```
4582
```

3.2 DistilBERT Fine-Tuning

I. Data Preparation and Train-Test Split

The same cleaned review texts and labels were used. The data was split into training and testing sets, identical to the BERT setup.

```
df_bert = pd.DataFrame({'text': final_cleaned, 'label': df['label']})

train_texts, test_texts, train_labels, test_labels = train_test_split(
    df_bert['text'].tolist(), df_bert['label'].tolist(), test_size=0.2, random_state=42
)
```

II. Tokenization

The DistilBERT tokenizer (distilbert-base-uncased) was used to tokenize the review texts, with padding and truncation to 128 tokens.

Tokenization DistilBERT

```
distil_tokenizer = DistilBertTokenizerFast.from_pretrained('distilbert-base-uncased')
train_encodings_distil = distil_tokenizer(train_texts, truncation=True, padding=True, max_length=128)
test_encodings_distil = distil_tokenizer(test_texts, truncation=True, padding=True, max_length=128)

train_dataset_distil = YelpDataset(train_encodings_distil, train_labels)
test_dataset_distil = YelpDataset(test_encodings_distil, test_labels)
```

III. Dataset Construction

The tokenized data and labels were wrapped into a custom PyTorch Dataset class for efficient data handling.

```
class YelpDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels
    def __getitem__(self, idx):
        return {key: torch.tensor(val[idx]) for key, val in self.encodings.items()} | {'labels': torch.tensor(self.labels[idx])}
    def __len__(self):
        return len(self.labels)

train_dataset = YelpDataset(train_encodings, train_labels)
test_dataset = YelpDataset(test_encodings, test_labels)
```

IV. Model Initialization and Fine-Tuning

The pre-trained DistilBERT model for sequence classification was loaded and configured for binary sentiment analysis. Training arguments were set, and the model was fine-tuned using the HuggingFace Trainer API.

DistilBERT model

```
distil_model = DistilBertForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=2)

if torch.cuda.is_available():
    torch.cuda.empty_cache()
```

```
distil_training_args = TrainingArguments(
    output_dir="./results_distil",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=64,
    eval_strategy="epoch",
    logging_dir="./logs_distil",
    logging_steps=10
)
```

Create Trainer

```
distil_trainer = Trainer(
    model=distil_model,
    args=distil_training_args,
    train_dataset=train_dataset_distil,
    eval_dataset=test_dataset_distil,
    compute_metrics=compute_metrics
)
```

DistilBERT Fine-Tuning Training

```
distil_trainer.train()
```

[150/150 00:34, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy
1	0.435700	0.496074	0.805000
2	0.164200	0.590135	0.765000
3	0.090200	0.595978	0.795000

```
TrainOutput(global_step=150, training_loss=0.3177455910046895, metrics={'train_runtime': 35.1508, 'train_samples_per_second': 68.277, 'train_steps_per_second': 4.267, 'total_flos': 161444642112000.0, 'train_loss': 0.3177455910046895, 'epoch': 3.0})
```

V. Evaluation and Metrics

After training, the DistilBERT model was evaluated on the test set. Metrics were calculated as follows:

- Accuracy: 80%
- Precision: 0.83
- Recall: 0.76
- F1 Score: 0.79

A confusion matrix was generated to visualize the results.

Evaluation

```
distil_eval_result = distil_trainer.evaluate()
run_time_distil = distil_eval_result['eval_runtime']
distil_acc = distil_eval_result['eval_accuracy']
results["DistilBERT Fine-Tuned"] = {"Accuracy": distil_acc}
print(f"\n DistilBERT Accuracy: {distil_eval_result['eval_accuracy']:.4f}")
```

```
DistilBERT Accuracy: 0.7950
```

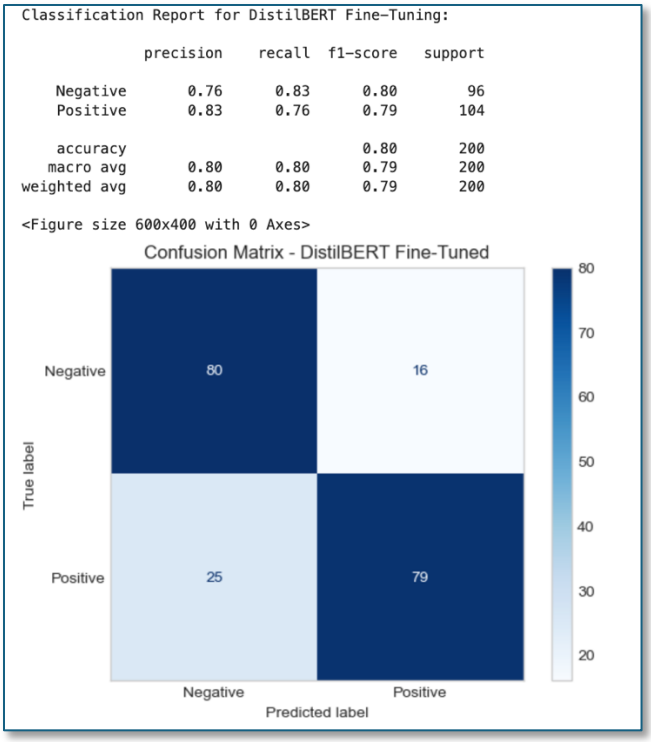
```
Classification Report for DistilBERT Fine-Tuning:

              precision    recall  f1-score   support

    Negative       0.76      0.83      0.80        96
    Positive       0.83      0.76      0.79       104

    accuracy                           0.80       200
   macro avg       0.80      0.80      0.79       200
weighted avg       0.80      0.80      0.79       200

<Figure size 600x400 with 0 Axes>
```



VI. Model Saving and Cleanup

The fine-tuned DistilBERT model and tokenizer were saved for future use. Memory was cleared.

Save DistilBERT Fine-Tuned model and tokenizer for future use

```
distil_trainer.save_model("./distilbert_saved_model")
distil_tokenizer.save_pretrained("./distilbert_saved_model")

del distil_trainer
del distil_model
del train_dataset_distil, test_dataset_distil
del train_encodings_distil, test_encodings_distil

if torch.cuda.is_available():
    torch.cuda.empty_cache()

gc.collect()
```

```
4775
```

3.3 RoBERTa Fine-Tuning

I. Data Preparation and Train-Test Split

The same cleaned review texts and labels were used, with the same train-test split as in previous models.

```
df_bert = pd.DataFrame({'text': final_cleaned, 'label': df['label']})

train_texts, test_texts, train_labels, test_labels = train_test_split(
    df_bert['text'].tolist(), df_bert['label'].tolist(), test_size=0.2, random_state=42
)
```

II. Tokenization

The RoBERTa tokenizer (roberta-base) was used to tokenize the review texts, with padding and truncation to 128 tokens.

Tokenization RoBERTa

```
roberta_tokenizer = RobertaTokenizerFast.from_pretrained('roberta-base')
train_encodings_roberta = roberta_tokenizer(train_texts, truncation=True, padding=True, max_length=128)
test_encodings_roberta = roberta_tokenizer(test_texts, truncation=True, padding=True, max_length=128)

train_dataset_roberta = YelpDataset(train_encodings_roberta, train_labels)
test_dataset_roberta = YelpDataset(test_encodings_roberta, test_labels)
```

III. Dataset Construction

The tokenized data and labels were wrapped into a custom PyTorch Dataset class.

```
class YelpDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels
    def __getitem__(self, idx):
        return {key: torch.tensor(val[idx]) for key, val in self.encodings.items()} | {'labels': torch.tensor(self.labels[idx])}
    def __len__(self):
        return len(self.labels)

train_dataset = YelpDataset(train_encodings, train_labels)
test_dataset = YelpDataset(test_encodings, test_labels)
```

IV. Model Initialization and Fine-Tuning

The pre-trained RoBERTa model for sequence classification was loaded and configured for binary sentiment analysis. Training arguments were set, and the model was fine-tuned using the HuggingFace Trainer API.

RoBERTa Model

```
roberta_model = RobertaForSequenceClassification.from_pretrained("roberta-base", num_labels=2)

if torch.cuda.is_available():
    torch.cuda.empty_cache()
```

```
roberta_training_args = TrainingArguments(
    output_dir="./results_roberta",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=64,
    eval_strategy="epoch",
    logging_dir="./logs_roberta",
    logging_steps=10
)
```

Create Trainer

```
roberta_trainer = Trainer(
    model=roberta_model,
    args=roberta_training_args,
    train_dataset=train_dataset_roberta,
    eval_dataset=test_dataset_roberta,
    compute_metrics=lambda pred: {"accuracy": accuracy_score(pred.label_ids, pred.predictions.argmax(-1))}
)
```

RoBERTa Fine-Tuning Training

```
roberta_trainer.train()
```

[150/150 01:05, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy
1	0.508800	0.536549	0.800000
2	0.314000	0.388868	0.830000
3	0.121400	0.462086	0.870000

```
TrainOutput(global_step=150, training_loss=0.38402488867441814, metrics={'train_runtime': 66.1169, 'train_samples_per_second': 36.299, 'train_steps_per_second': 2.269, 'total_flos': 295999937280000.0, 'train_loss': 0.384024888674
41814, 'epoch': 3.0})
```

V. Evaluation and Metrics

After training, the RoBERTa model was evaluated on the test set. Metrics were calculated as follows:

- Accuracy: 87%
- Precision: 0.89
- Recall: 0.86
- F1 Score: 0.87

A confusion matrix was generated to visualize the results.

Classsification report and Tokenizer of RoBERTa Fine-Tuning

```
roberta_preds_output = roberta_trainer.predict(test_dataset_roberta)
y_pred_roberta = roberta_preds_output.predictions.argmax(-1)
y_true_roberta = roberta_preds_output.label_ids

print("Classification Report for RoBERTa Fine-Tuning:\n")
print(classification_report(y_true_roberta, y_pred_roberta, target_names=["Negative", "Positive"]))
precision_roberta = precision_score(y_true_roberta,y_pred_roberta)
f1_roberta = f1_score(y_true_roberta,y_pred_roberta)
recall_roberta = recall_score(y_true_roberta,y_pred_roberta)

cm = confusion_matrix(y_true_roberta, y_pred_roberta)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Negative", "Positive"])

plt.figure(figsize=(6, 4))
disp.plot(cmap=plt.cm.Blues, values_format='d')
plt.title("Confusion Matrix - RoBERTa Fine-Tuned")
plt.grid(False)
plt.show()
```

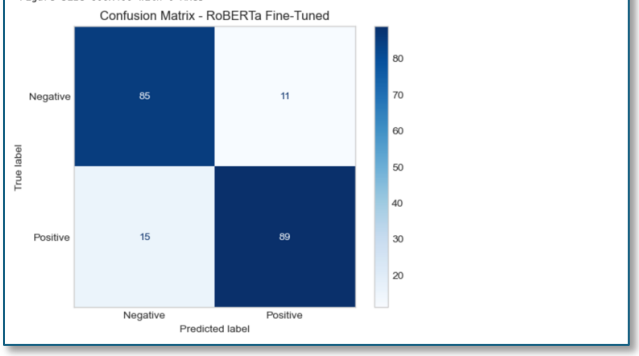
```
Classification Report for RoBERTa Fine-Tuning:

              precision    recall  f1-score   support

    Negative       0.85      0.89      0.87        96
    Positive       0.89      0.86      0.87       104

    accuracy                           0.87       200
   macro avg       0.87      0.87      0.87       200
weighted avg       0.87      0.87      0.87       200

<Figure size 600x400 with 0 Axes>
```



Evaluation of RoBERTa

```
roberta_eval_result = roberta_trainer.evaluate()
run_time_roberta = roberta_eval_result['eval_runtime']
roberta_acc = roberta_eval_result['eval_accuracy']
results["RoBERTa Fine-Tuned"] = {"Accuracy": roberta_acc}
print(f"\n RoBERTa Accuracy: {roberta_eval_result['eval_accuracy']:.4f}")
```

```
RoBERTa Accuracy: 0.8700
```

VI. Model Saving and Cleanup

The fine-tuned RoBERTa model and tokenizer were saved for future use. Memory was cleared.

Save RoBERTa Fine-Tuned model and tokenizer for future use

```
roberta_trainer.save_model("./roberta_saved_model")
roberta_tokenizer.save_pretrained("./roberta_saved_model")

del roberta_trainer
del roberta_model
del train_dataset_roberta, test_dataset_roberta
del train_encodings_roberta, test_encodings_roberta

if torch.cuda.is_available():
    torch.cuda.empty_cache()

gc.collect()
```

```
4497
```

All three transformer-based models outperformed traditional machine learning approaches on the Yelp sentiment classification task. BERT and RoBERTa achieved the highest accuracy (87%), while DistilBERT provided a good balance between speed and performance (80% accuracy). These results demonstrate the effectiveness of fine-tuned transformer models for text classification.

3.4 Model Comparison: Classical Machine Learning vs. Fine-Tuned Transformers

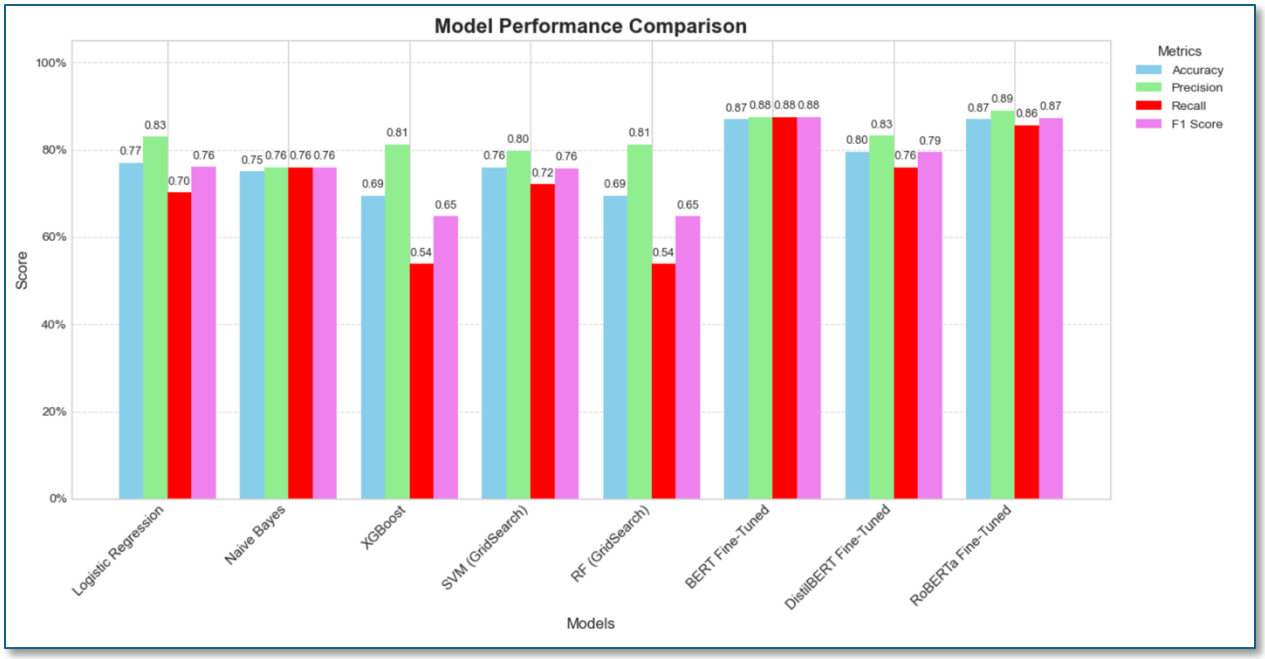
To figure out the best way to classify how people feel in Yelp reviews, I put some regular machine learning models head-to-head against newer transformer-based models. The regular models I used were Logistic Regression, Naive Bayes, XGBoost, SVM, and Random Forest. On the transformer side, I went with BERT, DistilBERT, and RoBERTa, making sure they were all trained on the same set of data.

I. Performance Metrics Comparison

The bar chart below shows how each model did in terms of accuracy, precision, recall, and F1 score. This makes it easy to see which models performed better overall.

- The traditional models like Logistic Regression and SVM did a decent job, with accuracy and F1 scores in the mid-70% range. Naive Bayes was also consistent, but XGBoost and Random Forest didn't really stand out for this dataset.
- The transformer models, on the other hand, clearly outperformed the classical ones. Both BERT and RoBERTa reached the highest accuracy (87%) and had strong F1 scores, showing they are much better at understanding the context and meaning in the reviews. DistilBERT was a bit less accurate but still did better than any of the traditional models, and it trained faster too.

This comparison really highlights how much of a difference transformer models can make for sentiment analysis, especially when you want the most accurate results.

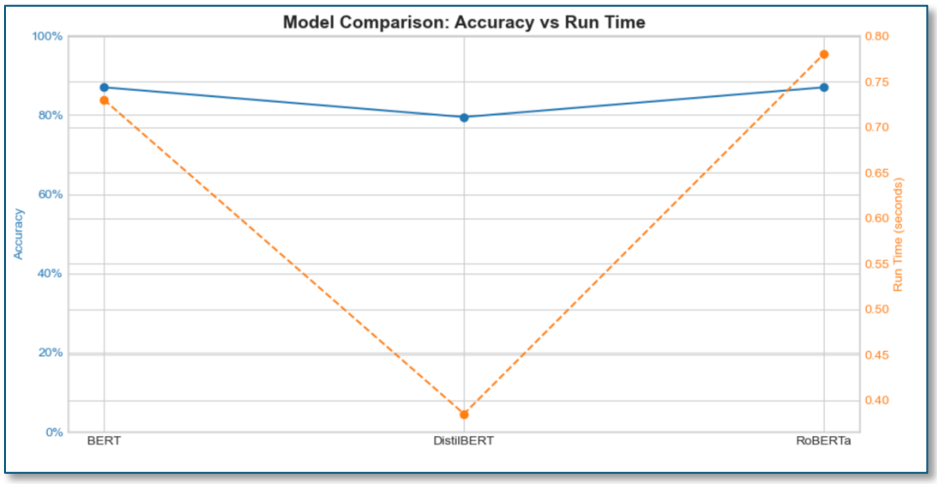


II. Accuracy vs. Run Time for Transformers

Besides accuracy, I also looked at how long each transformer model took to run. The line plot below compares the accuracy and run time for BERT, DistilBERT, and RoBERTa.

- BERT and RoBERTa both gave the best accuracy, but they also took the longest to train and evaluate because they are larger and more complex.
- DistilBERT, while a little less accurate, was much faster and used less computing power, making it a good option if you need results quickly or have limited resources.

This shows that there's a trade-off between getting the absolute best accuracy and having a model that's quick and efficient. Depending on your needs, you might choose one over the other.



Task 4: Topic Detection

In addition to sentiment classification, I wanted to understand what people talk about most in Yelp reviews. To do this, I used three popular topic modeling techniques: Latent Dirichlet Allocation (LDA), Non-negative Matrix Factorization (NMF), and Latent Semantic Analysis (LSA). These methods help uncover the main themes by grouping together words that often appear in similar contexts.

- **LDA - Latent Dirichlet Allocation**
- **NMF - Non-negative Matrix Factorization**
- **LSA - Latent Semantic Analysis**

4.1 Data Preparation for Topic Modeling

First, I used the final cleaned text from the preprocessing steps. To turn the text into a format suitable for topic modeling, I used two different vectorizers:

- CountVectorizer for LDA, which counts how often each word appears in the reviews.
- TfidfVectorizer for NMF and LSA, which considers both how often a word appears and how unique it is across all reviews.

This step resulted in document-term matrices that represent the text data numerically.

CountVectorizer for LDA

```
count_vectorizer = CountVectorizer(max_df=0.95, min_df=2, stop_words='english')
count_dtm = count_vectorizer.fit_transform(final_cleaned)
```

TfidfVectorizer for NMF

```
tfidf_vectorizer = TfidfVectorizer(max_df=0.95, min_df=2, stop_words='english')
tfidf = tfidf_vectorizer.fit_transform(final_cleaned)
```

TfidfVectorizer for LSA

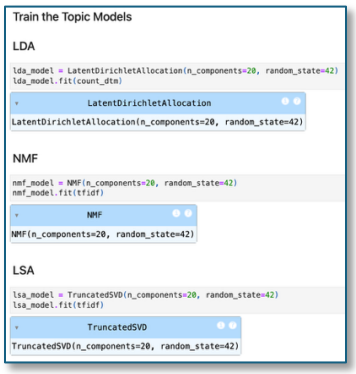
```
vectorizer = TfidfVectorizer(max_df=0.95, min_df=2, stop_words='english')
tfidf = vectorizer.fit_transform(final_cleaned)
feature_names = vectorizer.get_feature_names_out()
```

4.2 Training the Topic Models

I trained each topic model to find 20 topics in the Yelp reviews:

- LDA was trained on the count-based matrix.
- NMF and LSA were trained on the TF-IDF matrix.

Each model learned to group together words that frequently appear together, which can be interpreted as topics or themes in the reviews.



4.3 Extracting and Interpreting Topics

For each model, I extracted the top 20 words for each topic. These words give a sense of what each topic is about. For example, in LDA, one topic included words like "service," "slow," "wait," and "excellent," which likely relates to customer service experiences. Another topic included "pizza," "crust," "cheese," and "tasty," pointing to food quality.

Here are some sample topics from each model:

LDA Topics:

```
--- LDA Topics ---

Topic 1: way | cold | overpriced | chicken | friend | special | hot | barely | like | good | food | dish | salad | perfect | damn | bad | feel | fried | dressing | greek
Topic 2: really | experience | place | good | great | know | food | atmosphere | hour | seriously | need | menu | establishment | definitely | fish | overpriced | pizza | got | like | long
Topic 3: place | like | eat | sushi | great | love | time | want | restaurant | good | vega | avoid | authentic | healthy | steak | cooked | better | potato | believe | party
Topic 4: night | come | review | right | rude | fry | burger | got | place | asked | management | disappointment | perfect | seated | really | vega | came | time | eating | away
Topic 5: going | soon | getting | anytime | say | try | pretty | think | food | seafood | tasteless | felt | insulted | taste | like | place | portion | generous | disrespected | gross
Topic 6: time | food | good | best | buffet | vega | breakfast | tasty | check | waste | really | ordered | bland | far | year | average | twice | spot | ago | considering
Topic 7: great | food | service | server | nice | like | time | bar | ambiance | quality | feel | fantastic | menu | attentive | wrong | clean | came | table | staff | thing
Topic 8: service | slow | wait | food | awesome | bad | excellent | time | came | lunch | vega | people | customer | fantastic | pleased | rude | restaurant | going | best | worth
Topic 9: delicious | time | think | food | dinner | absolutely | gone | bartender | great | went | got | dining | taste | business | multiple | deal | meh | bother | fresh | longer
Topic 10: disappointed | place | hard | overall | money | happy | say | horrible | server | time | quality | think | restaurant | friendly | husband | owner | twice | wait | impressed | kind
Topic 11: star | place | recommend | experience | time | said | coming | disappointing | zero | sauce | wonderful | deserves | location | restaurant | best | salad | food | egg | roll | lobster
Topic 12: fresh | waited | lot | atmosphere | great | like | food | perfect | fun | family | biscuit | steak | pay | small | pizza | definitely | chicken | recommended | price | fry
Topic 13: service | food | friendly | terrible | good | want | waitress | chicken | super | mediocre | manager | better | little | taco | make | outstanding | spot | server | bad | talk
Topic 14: staff | selection | great | probably | beer | friendly | food | good | incredible | dessert | super | ordered | sucked | ice | hit | love | day | yummy | small | unfortunately
Topic 15: love | tried | salad | like | table | food | quite | day | bean | dish | green | job | large | chip | disappointed | good | tell | way | left | fry
Topic 16: meal | menu | bland | people | order | potato | flavor | bit | thing | sweet | drive | subway | fact | option | worth | disappointed | good | waitress | pretty | really
Topic 17: worst | nice | flavor | sandwich | taste | sauce | way | reasonable | delicious | spicy | dry | meat | pork | wow | price | inside | went | tender | trip | frozen
Topic 18: amazing | minute | place | say | food | dish | drink | took | eating | pizza | pho | word | waited | deal | burger | delicious | tasted | salmon | expected | walked
Topic 19: good | place | food | pretty | price | loved | got | eat | impressed | bite | quick | beef | burger | came | little | phoenix | minute | table | pizza | inside
Topic 20: best | steak | restaurant | definitely | really | beat | town | sure | better | expect | visit | shrimp | ordered | time | buffet | fantastic | tasted | new | worse | hour
```

NMF Topics:

```
--- NMF Topics ---

Topic 1: good | price | selection | pizza | tasty | stuff | thing | unfortunately | soggy | money | crust | tea | way | thought | expect | joint | pretty | waitress | beer | twice
Topic 2: food | terrible | mediocre | average | quality | bar | got | came | wasting | say | wrong | omg | order | impressed | small | looking | bland | meh | overall | horrible
Topic 3: place | recommend | love | eat | night | loved | sushi | authentic | overpriced | avoid | recommended | want | lot | thumb | vega | phoenix | gyro | disgusting | heart | glad
Topic 4: service | slow | fantastic | terrible | excellent | poor | customer | suck | attentive | friendly | quick | lot | extremely | cute | boot | awful | fast | par | meh | came
Topic 5: great | eat | menu | dinner | price | atmosphere | fry | server | touch | portion | pizza | wine | finish | drink | setting | bay | value | huge | chef | salad
Topic 6: time | waste | wait | money | vega | think | soon | second | minute | server | multiple | went | came | coming | horrible | new | lunch | got | kept | asked
Topic 7: like | feel | thing | overall | order | know | probably | better | blow | quick | dessert | chicken | felt | dark | crazy | dish | fry | dirt | warm | mary
Topic 8: delicious | absolutely | fresh | spicy | cocktail | dish | wow | deal | tapa | evening | wonderful | preparing | watch | taste | way | bartender | twice | different | loved | ask
Topic 9: friendly | staff | waitress | super | attentive | nice | clean | waiter | server | quick | ambiance | joint | helpful | owner | manager | stopped | happy | kind | fine | option
Topic 10: amazing | say | absolutely | burger | dessert | chip | pizza | taste | pho | bartender | second | vegetarian | menu | cow | taco | meal | today | interesting | later | opportunity
Topic 11: restaurant | love | wait | excellent | quality | atmosphere | know | wonderful | nicest | little | family | make | excuse | thing | oh | bay | inexpensive | new | chinese | lost
Topic 12: definitely | come | recommend | worth | eat | soon | pizza | vega | establishment | paid | unless | wing | bar | fresh | doubt | chicken | taste | spot | hit | seafood
Topic 13: disappointed | try | assure | burger | tell | husband | ordered | way | order | overall | quality | drive | bit | coffee | hate | need | point | cafe | bay | quite
Topic 14: experience | disappointing | pretty | fun | overall | lunch | dining | better | make | tasty | wonderful | location | hope | recent | town | try | liked | want | happy | look
Topic 15: best | steak | buffet | vega | breakfast | far | town | selection | average | tot | bar | cheese | price | beat | kind | entree | check | shrimp | taco | seafood
Topic 16: going | awesome | think | soon | anytime | meal | beer | pretty | selection | say | try | business | fact | needless | getting | offer | regular | quickly | hair | want
Topic 17: worst | tasted | probably | eat | dish | manager | thai | sushi | people | selection | fry | boot | frozen | salmon | sashimi | seen | coming | doubt | dirt | honest
Topic 18: really | nice | atmosphere | better | owner | enjoyed | sandwich | eating | menu | rice | drink | people | meal | impressed | station | inside | chip | spicy | fast | friend
Topic 19: bad | flavor | treated | cold | rude | customer | lunch | damn | people | friend | barely | pasta | bland | said | suck | husband | heart | little | finish | world
Topic 20: star | deserves | zero | possible | gave | review | eat | appetizer | started | given | eating | bread | hard | burger | hot | fact | know | vibe | brick | oven
```

LSA Topics:

```
--- LSA Topics ---

Topic 1: good | food | service | great | place | time | really | price | friendly | like | best | terrible | bad | selection | amazing | delicious | awesome | say | experience | eat
Topic 2: good | selection | price | stuff | tasty | pizza | soggy | expect | unfortunately | crust | joint | tea | real | bacon | looking | close | beef | red | flavorless | sweet
Topic 3: place | good | time | really | recommend | love | eat | amazing | night | sushi | vega | pretty | loved | thing | town | like | overpriced | come | authentic | avoid
Topic 4: service | place | good | friendly | slow | like | fantastic | recommend | poor | nice | waitress | excellent | better | pretty | quick | lot | customer | suck | star | super
Topic 5: great | good | time | experience | staff | friendly | pizza | dinner | atmosphere | menu | eat | lunch | price | think | fry | server | steak | touch | wine | spot
Topic 6: time | like | delicious | bad | think | waste | experience | soon | wait | staff | vega | going | minute | restaurant | came | second | money | lunch | amazing | coming
Topic 7: like | really | restaurant | feel | thing | overall | better | great | know | probably | order | tasted | fry | chicken | blow | people | quick | quite | friend | dish
Topic 8: delicious | like | experience | fresh | absolutely | dish | spicy | amazing | cocktail | taste | deal | great | know | wow | tapa | wonderful | worst | good | nice | better
Topic 9: friendly | staff | amazing | waitress | super | nice | say | disappointed | attentive | burger | clean | waiter | absolutely | server | dessert | ambiance | quick | joint | helpful | taco
Topic 10: amazing | disappointed | burger | awesome | dessert | say | absolutely | service | pizza | meal | chip | taste | came | menu | pho | second | like | going | vegetarian | great
Topic 11: definitely | best | really | restaurant | worst | experience | vega | come | steak | disappointed | amazing | buffet | worth | atmosphere | better | love | town | breakfast | menu | price
Topic 12: disappointed | experience | worst | bad | overall | try | really | disappointing | lunch | pretty | love | husband | bit | assure | tell | impressed | quality | way | think | order
Topic 13: disappointed | definitely | going | soon | delicious | think | minute | eat | anytime | come | awesome | wait | star | try | order | menu | ordered | recommend | restaurant | burger
Topic 14: definitely | experience | worst | awesome | going | eat | bad | friendly | pretty | recommend | soon | like | staff | disappointing | star | come | selection | beer | anytime | say
Topic 15: best | worst | selection | steak | buffet | far | breakfast | eat | vega | town | tasted | average | star | like | thing | salmon | dish | beer | tot | cheese
Topic 16: going | awesome | restaurant | think | star | pretty | love | soon | anytime | excellent | selection | meal | say | experience | worst | vega | beer | wait | minute | getting
Topic 17: worst | terrible | eat | really | tasted | fantastic | wait | probably | dish | atmosphere | time | thai | nice | server | way | mediocre | boot | manager | star | frozen
Topic 18: really | going | awesome | star | meal | steak | best | think | bad | friend | sandwich | eating | fresh | flavor | enjoyed | people | menu | chicken | time | deserves
Topic 19: star | eat | restaurant | deserves | bad | zero | review | amazing | atmosphere | know | gave | possible | experience | eating | absolutely | vibe | appetizer | seriously | hard | family
Topic 20: love | bad | star | flavor | way | chicken | salad | fry | overpriced | selection | worst | cold | bland | burger | meat | deserves | pretty | eat | zero | worth
```

4.4 Visualizing Topics

To make the topics easier to understand, I created bar plots showing the most important words in the top five topics for each model. These visualizations help quickly identify the main themes discussed in the reviews, such as food quality, service, pricing, and atmosphere.

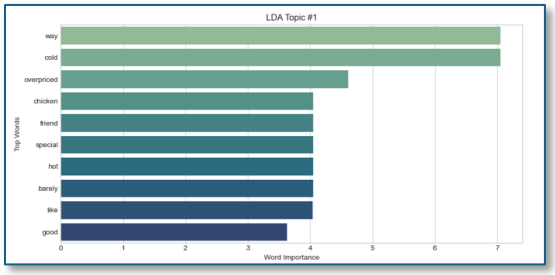
Plot Top 5 Topics from Each Model

```python
def plot_top_words(model, feature_names=None, n_top_words=10, n_topics=5, model_type='lsa'):
    print(f"\n Plotting {n_topics} Topics from {model_type.upper()}...\n")

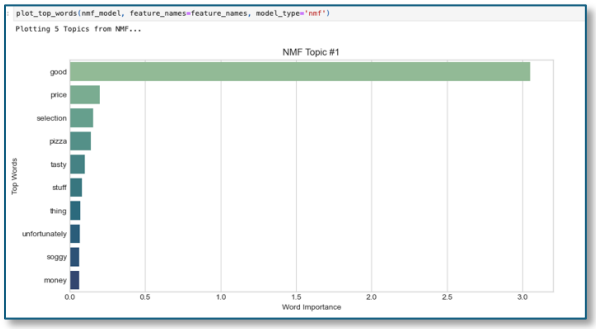
    if model_type in ['lsa', 'nmf', 'lda']:  # all sklearn models
        for topic_idx in range(n_topics):
            topic = model.components_[topic_idx]
            top_indices = topic.argsort()[:-n_top_words - 1:-1]
            top_words = [feature_names[i] for i in top_indices]
            top_scores = topic[top_indices]

            plt.figure(figsize=(10, 5))
            sns.barplot(x=top_scores, y=top_words, palette='crest',hue=top_words)
            plt.title(f"{model_type.upper()} Topic #{topic_idx + 1}")
            plt.xlabel("Word Importance")
            plt.ylabel("Top Words")
            plt.tight_layout()
            plt.show()
```

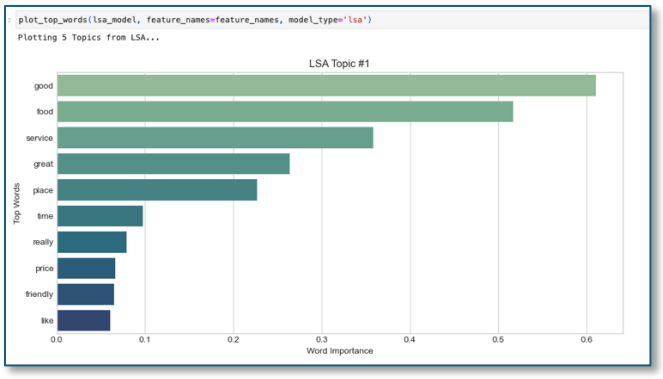
LDA Topic 1:



NMF Topic 1:

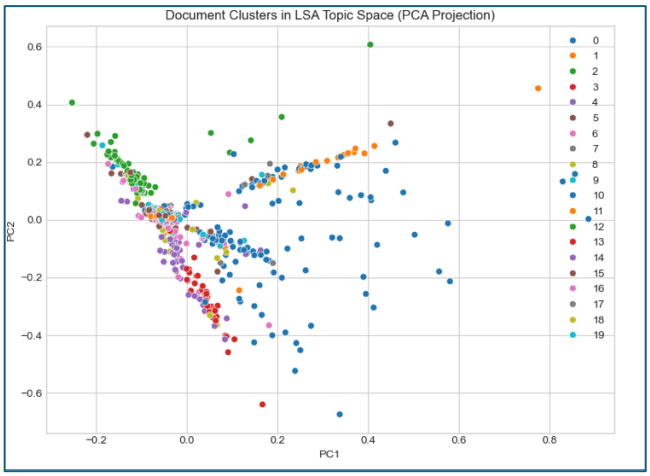


LSA Topic 1:



4.5 Document Clustering in Topic Space

To see how reviews are grouped by topic, I used Principal Component Analysis (PCA) to reduce the topic distributions to two dimensions. I then plotted the reviews in this topic space, coloring each point by its dominant topic. This scatter plot shows how reviews cluster together based on what they talk about most.



4.6 Differences Between Methods

Topic Coherence: LDA often produced topics that were more interpretable and focused, with top words in each topic clearly relating to a single theme (e.g., a topic about service, another about food quality). NMF also produced coherent topics but sometimes included a mix of related and less-related words. LSA, on the other hand, sometimes generated topics that were harder to interpret, with top words that did not always form a clear theme.

Topic Overlap: LSA tended to have more overlap between topics, with some words appearing in multiple topics. LDA and NMF generally produced more distinct topics, making it easier to assign a clear label to each one.

Sensitivity to Rare Words: NMF and LSA, which use TF-IDF, sometimes highlighted more unique or less frequent words, while LDA (using raw counts) focused more on the most common words in the dataset.

Interpretability: In practice, I found LDA topics the easiest to interpret, followed by NMF. LSA was useful for dimensionality reduction and clustering, but its topics were sometimes less intuitive.

4.7 Findings

Topic modeling revealed several recurring themes in the Yelp reviews, such as food quality, service experience, pricing, and atmosphere. Each method (LDA, NMF, LSA) produced slightly different groupings, but all helped uncover the main concerns and highlights mentioned by customers. These insights can be valuable for restaurant owners or analysts who want to understand what matters most to their customers beyond just positive or negative sentiment.

References

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

https://scikit-learn.org/

Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020: System Demonstrations, 38–45.

https://huggingface.co/transformers/

Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media Inc.

https://www.nltk.org/

Chollet, F. (2015). Keras.

https://keras.io/

Yelp Dataset. UCI Machine Learning Repository.

https://archive.ics.uci.edu/ml/datasets/sentiment+labelled+sentences

Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. Journal of Machine Learning Research, 3, 993–1022.

https://www.jmlr.org/papers/volume3/blei03a/blei03a.pdf

Lee, D. D., & Seung, H. S. (1999). Learning the parts of objects by non-negative matrix factorization. Nature, 401(6755), 788–791.

https://www.nature.com/articles/44565

Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by Latent Semantic Analysis. Journal of the American Society for Information Science, 41(6), 391–407.

https://www.researchgate.net/publication/220734964_Indexing_by_latent_semantic_analysis

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD '16, 785–794.

https://dl.acm.org/doi/10.1145/2939672.2939785

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019, 4171–4186.

https://arxiv.org/abs/1810.04805

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.

https://arxiv.org/abs/1910.01108

Liu, Y., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach.

https://arxiv.org/abs/1907.11692

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51–56.

https://pandas.pydata.org/

Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357–362.

https://numpy.org/

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.

https://matplotlib.org/